

ANGULAR FULL-STACK MASTERCLASS

課程目錄

點擊章節卡片開始學習

Ch 1

前端開發介紹

Frontend Intro

Ch 2

Coding 習慣

Best Practices

Ch 3

Angular 介紹

Angular Essentials

Ch 4

終端機指令

Terminal / CMD

Ch 5

安裝 Angular

Setup & New Project

Ch 6

Angular 降版

Version Downgrade

Ch 7

HTML 基礎語法

HTML Essentials

Ch 8

安裝 VS Code

Install VS Code

FRONTEND DEVELOPMENT FUNDAMENTALS

前端介紹

「大眾媒介 - 網頁」

[← 返回目錄](#)

Outline

- 大眾媒介 - 網頁：前後端與資料庫的關係
- 何謂前端？
- 前端三大技術：HTML、CSS/SCSS、TypeScript
- 各技術介紹
- 學習路線建議

大眾媒介

網頁的世界

大眾媒介 - 網頁 (Web)

網頁能在各種不同裝置上顯示介面，其背後是由三大核心組成的資料流程。

前端 Frontend

- 服務回傳的資料呈現
- 最直接的顯示介面
- 使用者操作的裝置

後端 Backend

- 提供網頁服務
- 接收資料並處理
- 連接各資料庫

資料庫 Database

- 儲存使用者資訊
- 增刪查改資料
- 紀錄資訊

💡 **資料流：** 前端接收資料並傳遞 → 後端處理完資料並回傳 → 前端顯示給使用者

前端與後端的互動

- 前端 (Frontend) :
 - 顯示使用者選用任一服務時所呈現的資料
 - 使用者操作的裝置 (Phone / PC / Pad / Watch 等)
- 後端 (Backend) :
 - 接收前端的服務請求並傳回相對應的資料
 - 與資料庫連接，進行資料的增加、刪除、查詢與修改

💡 重點：多種大小的裝置皆不同，但網頁能在各種不同介面提供服務。網頁服務分為前端、後端兩部分。

何謂前端？

What is Frontend?

何謂前端？

前端主要在提供的介面上顯示內容針對使用者。

面向	說明
多樣載體	Phone / PC / Pad / Watch 等
開發語法	Swift (iOS)、JAVA (Android)、Angular (跨平台)
設計概念	每個顯示的頁面都經過 UI / UX 設計與對應功能連結
核心目標	讓使用者在啟用服務時能夠有良好的體驗

💡 前端的目的是：不必讓使用者面對死板板的程式碼，而是提供友善的操作介面。

前端的建築師們

HTML、CSS/SCSS、TypeScript

前端三大技術概覽

技術	比喻	特點
HTML	網頁的骨架	不算程式語言，屬樣板語言；需硬記語法，邏輯需求不高
SCSS / CSS	網頁的衣服	負責排版、顏色；HTML 結構相關知識；輸出全靠死背！
TS (TypeScript)	網頁的動作	考驗邏輯能力，比 JavaScript 嚴謹；好的邏輯架構如魚得水

HTML - 網頁的骨架

負責內容結構，不算程式語言，屬於樣板語言。

- 透過 `<link>` 引入 `style.css`，透過 `<script>` 引入 `main.js`

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <link rel="stylesheet" href="style.css">
5 </head>
6 <body>
7   <button id="greet-btn">打招呼</button>
8   <p id="message"></p>
9   <script src="main.js"></script>
10
11 </body>
12
13 </html>
```

CSS / SCSS - 網頁的衣服

負責排版、顏色、字型與響應式設計 (RWD)。

- 針對 HTML 中的 `#greet-btn` 套上顏色、間距與圓角

```
1  /* style.css */
2  #greet-btn {
3      background-color: #5eada0;
4      color: white;
5      border: none;
6      padding: 10px 20px;
7      border-radius: 8px;
8      cursor: pointer;
9      transition: all 0.3s ease;
10 }
```

TypeScript - 網頁的動作

負責互動與邏輯，讓頁面能回應使用者的操作。

- 監聽 `#greet-btn` 的點擊事件，點擊後更新 `#message` 的文字

```
1 // main.js
2 const btn = document.getElementById('greet-btn');
3 const msg = document.getElementById('message');
4
5 btn.addEventListener('click', () => {
6   msg.textContent = '你好！歡迎來到前端的世界！';
7 });
```

學習路線建議

Learning Path

學習路線建議

正確的學習順序能讓你事半功倍：

HTML → CSS → JavaScript → 框架 (Angular / React / Vue)

階段	技術	原因
第一步	HTML	所有框架的基礎架構
第二步	CSS / SCSS	讓頁面有樣式
第三步	JavaScript / TypeScript	加入互動與邏輯
第四步	框架 (Angular 等)	基於前三者延伸

💡 為什麼要先學 HTML？因為市面上所有的框架都是基於 HTML 的架構去做延伸，HTML 就是三大框架的基礎。

介紹結束

準備好開始你的前端之旅了嗎？

PROGRAMMING BEST PRACTICES

Coding 習慣

「好的習慣，讓程式開發更快速、維護更輕鬆」

[← 返回目錄](#)

Outline

- 1. 命名方式 — 清楚命名，讓人一眼看懂
- 2. 檔案放置位置 — 結構統一，方便維護
- 3. 排版 — 整潔排版，提升可讀性
- 4. 註解 — 留下脈絡，方便接手
- 5. 避免重複程式碼 — DRY 原則，減少錯誤

1. 命名方式

Naming Convention

命名方式

原則	說明
清楚易懂	讓人一眼看出這個變數 / 方法的用途
避免自創縮寫	真的需要縮寫，請加上註解說明
使用有意義的名稱	例如：使用者名稱用 <code>userName</code> ，不用 <code>un</code>
團隊統一風格	可根據開發團隊討論決定命名規則

💡 **重點：** 命名不只是自己看，更是讓一起開發與維護的夥伴能一眼看清楚這是什麼東西。

命名方式 – 範例

```
1 // ❌ 不好的命名：縮寫不明，難以理解
2 let un = "Allen";
3 let pw = "123456";
4 function calc(a: number, b: number) { return a + b; }
5
6 // ✅ 好的命名：清楚表達用途
7 let userName = "Allen";
8 let password = "123456";
9 function calculateTotal(price: number, tax: number) { return price + tax; }
```

2. 檔案放置位置

File Structure / Layout

檔案放置位置

原則	說明
分類放置	同類型檔案放在對應資料夾中
統一結構 (Layout)	專案裡有許多檔案，需統一擺放規則
範例	RESTful API 相關檔案放在 <code>api/</code> 資料夾
以團隊為主	以團隊開發的便利性為主，統一討論決定

💡 **為什麼重要：** 若每個人隨意擺放，維護時極高機率找不到檔案或少修改某隻檔案，導致系統出現問題。

3. 排版

Code Formatting

排版 – 未整理的程式碼

```
1  int main() {printf("Hello World");  
2  return 0;}
```

✗ 難以閱讀：程式碼擠在一起，沒有縮排，除錯與維護時容易出錯。

排版 — 整理後的程式碼

```
1  int main() {  
2      printf("Hello World");  
3      return 0;  
4  }
```

✓ 易於閱讀：縮排清楚、結構分明，維護與除錯都更方便。程式越大，良好排版的重要性越高。

4. 註解

Code Comments

註解的用途

用途	說明
程式出處	記錄這段程式碼的來源或背景
待處理問題	開發中尚未解決的 TODO 事項
參考資料	記錄參考的文件或網址
開發者資訊	誰開發了哪段程式碼與開發日期

⚠ 注意：註解要讓所有人都能理解，而不是只有自己看得懂，否則就失去了寫註解的意義。

註解 — 範例

```
1 // TODO: 待串接後端 API，目前先回傳假資料 (Allen, 2024-01-15)
2 getUserList(): User[] {
3     return mockUsers;
4 }
5
6 // 參考: Angular 官方文件 https://angular.io/guide/http
7 // 此方法負責處理 HTTP 錯誤回應
8 handleError(error: HttpResponse): Observable<never> {
9     console.error('API Error:', error.message);
10
11     return throwError(() => error);
12 }
```

5. 避免重複程式碼

DRY Principle

避免重複程式碼 – 問題

當相同 (或 8~9 成相似) 的程式碼出現在多個地方，代表程式碼不健康。

```
1 // ❌ 複製貼上：每個地方都有相同邏輯  
2 calculatePriceA() { const tax = price * 0.05; return price + tax; }  
3 calculatePriceB() { const tax = price * 0.05; return price + tax; }  
4 calculatePriceC() { const tax = price * 0.05; return price + tax; }
```

❌ 問題：若邏輯有誤或需要修改，需要同時改多個地方，很容易遺漏，導致系統出現問題。

避免重複程式碼 — 解法

發現相同邏輯時，將其提取為共用方法，各處呼叫即可。

```
1  // ✅ 提取為共用方法
2  calculateTax(price: number): number {
3    return price * 0.05;
4  }
5
6  // 各處呼叫共用方法
7  calculatePriceA(price: number) { return price + this.calculateTax(pric
e); }
8  calculatePriceB(price: number) { return price + this.calculateTax(pric
e); }
```

✅ **DRY 原則**：Don't Repeat Yourself — 只需修改一處，所有地方同步更新，大幅降低出錯機率。

課程結束

養成好習慣，寫出易讀、易維護的程式碼！

ANGULAR ESSENTIALS

Angular 介紹

「一個整合了路由、表單、通訊的全功能前端開發平台」

[← 返回目錄](#)

Outline

- 1. **Angular** 是什麼 — 平台定義與三大組成
- 2. 為什麼選 **Angular** — 三大框架比較、優缺點分析
- 3. 前置學習需求 — HTML/CSS/JS 基礎、Node.js、npm

1. Angular 是什麼？

What is Angular?

Angular 是什麼？

Angular 是一個基於 **TypeScript** 的開發平台，不只是框架，而是一套完整的工具生態系。

組成要素	說明
元件化框架	建構可延展的 Web 應用程式
整合函式庫	路由機制、表單管理、Client/Server 通訊等
開發工具	幫助開發、建置、測試、更新程式碼

💡 簡單來說：Angular 是一個幫你整合了許多工具的網頁開發平台。

2. 為什麼選 Angular ?

Why Angular?

三大前端框架比較

目前主流三大前端框架：Vue、React、Angular。

框架	學習難度	靈活性	規範程度
Vue	★ 最低	高	寬鬆
React	★★ 適中	高	寬鬆
Angular	★★★ 最高	較低	嚴謹

💡 為什麼選 **Angular**：雖然學習難度最高，但其嚴謹規範能有效降低後續維護成本，適合團隊協作。

Angular 優點

優點

說明

適合各規模專案

大型 / 小型專案皆可使用

規範嚴謹

統一的寫法，降低後續維護成本

官方文件齊全

文件完整，討論社群活躍

Angular 缺點

缺點	說明
學習難度高	相較 Vue、React 學習曲線較陡
彈性較低	嚴格規範，開發方式較不靈活
升級較為困難	版本升級複雜（官方持續優化中）

⚠️ **注意：**缺點中「升級較困難」是相對於其他框架而言，Angular 官方也在持續改善升級體驗。

3. 前置學習需求

Prerequisites

前置知識需求

學習 Angular 前，需先具備以下基礎：

知識領域	說明
HTML	網頁結構語言，負責畫面骨架
CSS	網頁樣式語言，負責畫面外觀
JavaScript	前端程式語言基礎
終端機 / 命令提示字元	基本指令操作能力

Node.js

項目	說明
全名	Node.js
類型	開源的 JavaScript 運行環境
用途	允許開發者在 伺服器端 運行 JavaScript (TypeScript 也適用)
為何需要	Angular CLI 與開發工具建置在 Node.js 環境上

💡 **重點：** Node.js 讓 JavaScript 不只能在瀏覽器執行，也能在本機環境執行開發工具。

npm (Node Package Manager)

項目	說明
全名	Node Package Manager
字面意義	管理 Node 套件的工具
用途	提供開發者分享、發布、管理 Node.js 模組的平台與工具
安裝方式	隨 Node.js 一起安裝，無需另外安裝

```
1 # 確認安裝版本
2 node -v
3 npm -v
```

💡 關係：安裝 Node.js 時會自動附帶 npm，兩者一起安裝。

課程結束

接下來：安裝 **Angular** 開發環境，開始實作！

終端機 / 命令提示字元

「前端開發必備工具，用指令控制你的電腦」

[← 返回目錄](#)

Outline

- 1. 什麼是終端機 — Mac/Linux 終端機 vs Windows 命令提示字元
- 2. 常用指令對照 — MacOS/Linux 與 Windows 指令比較
- 3. **cd** 指令詳解 — 目錄切換的各種用法

1. 什麼是終端機？

Terminal / Command Prompt

什麼是終端機？

開發前端時一定會使用到的工具，用來透過**文字指令**操作電腦。

作業系統	工具名稱	底層系統
macOS	終端機 (Terminal)	Linux 系統
Windows	命令提示字元 (cmd)	Windows

💡 為什麼要學：Angular CLI、npm 指令等開發工具都需要透過終端機執行，是前端開發的必備技能。

2. 常用指令對照

Common Commands

常用指令對照表

說明	macOS / Linux	Windows
切換目錄	<code>cd</code>	<code>cd</code>
取得目前所在位置	<code>pwd</code>	<code>chdir</code>
列出目前的檔案列表	<code>ls</code>	<code>dir</code>
刪除檔案	<code>rm</code>	<code>del</code>

常用指令 – 範例 (macOS / Linux)

```
1 # 切換到 projects 目錄
2 cd projects
3
4 # 查看目前位置
5 pwd
6
7 # 列出所有檔案
8 ls
9
1
0 # 刪除檔案
1
1 rm old-file.txt
```

常用指令 – 範例 (Windows)

```
1 # 切換到 projects 目錄
2 cd projects
3
4 # 查看目前位置
5 chdir
6
7 # 列出所有檔案
8 dir
9
1
0 # 刪除檔案
1
1 del old-file.txt
```

3. cd 指令详解

Directory Navigation

cd 指令用法

`cd` (change directory) 是最常用的指令，用來切換目錄位置。

指令	說明
<code>cd ..</code>	返回上一層目錄
<code>cd ~</code>	返回家目錄 (最上層，macOS/Linux)
<code>cd 路徑</code>	移動到指定的路徑
<code>D:</code>	切換到 D 槽 (Windows 切換硬碟專用)

⚠ **Windows 注意：** 要切換到其他硬碟時，不能用 `cd`，需直接輸入硬碟名稱，例如：`D:`

cd 指令 – 範例

```
1 # 返回上一層
2 cd ..
3
4 # 返回家目錄 (macOS/Linux)
5 cd ~
6
7 # 移動到指定路徑
8 cd C:/Users/allen/projects/my-app
9
1
0 # Windows：切換到 D 槽
1
1 D:
```

課程結束

熟悉終端機指令，開始使用 **Angular CLI** !

ANGULAR ESSENTIALS

安裝 / 新建 Angular 專案

「從零開始，建立你的第一個 Angular 專案」

[← 返回目錄](#)

Outline

- 1. 安裝前置工具 — NVM、Node.js、Angular CLI
- 2. 建立與執行專案 — ng new、ng serve
- 3. 認識專案結構 — 根目錄、src 資料夾、元件組成

1. 安裝前置工具

Install Prerequisites

什麼是 NVM ?

開發時有時候會遇到不同版本的 **Node.js 專案**，不同版本有時候會有不同運作方式。

工具	說明
NVM	Node Version Manager，讓同一台電腦安裝多個 Node.js 版本
用途	依專案需求自由切換 Node.js 版本

💡 安裝順序：NVM → Node.js → Angular CLI，需依序安裝。

安裝 NVM (Windows)

前往以下網址，下載最新版本的 `nvm-setup` 並安裝：

```
1 # 網址  
2 https://github.com/coreybutler/nvm-windows/releases
```

安裝完成後，開啟命令提示字元並輸入 `nvm`，若看到說明畫面即安裝成功。

💡 下載選項：選擇 `nvm-setup`（安裝版），不要選 `nvm-noinstall`。

安裝 NVM (macOS)

macOS 安裝較為複雜，需至 NVM 官方 GitHub 依說明步驟安裝：

```
1 # 官方 GitHub  
2 https://github.com/nvm-sh/nvm
```

⚠ 注意：macOS 需依照 GitHub 上的說明一步一步執行，安裝後可能需要重新開啟終端機。

安裝 Node.js

NVM 安裝完成後，用它來安裝 Node.js。建議先至官方確認目前的 LTS (長期支援) 穩定版本。

```
1 # 安裝指定版本 (例如安裝 22.x 最新版)
2 nvm install 22
3
4 # 確認安裝成功
5 node -v
```

💡 技巧：只輸入主版本號 (如 **22**)，NVM 會自動安裝該版本最新的小版號。若 `node -v` 無反應，關閉命令提示字元重開再試。

安裝 @angular/cli

Node.js 安裝完成後，用 npm 全域安裝 Angular CLI：

```
1 # 安裝 Angular CLI
2 npm install -g @angular/cli@19.0.5
3
4 # 確認安裝成功
5 ng v
```

💡 確認成功：輸入 `ng v` 後若看到版本清單，即代表安裝成功，之後即可使用 `ng` 指令。

2. 建立與執行專案

Create & Run Project

建立 Angular 專案

開啟命令提示字元，切換至想放置專案的目錄，再執行 `ng new`：

```
1 # 建立新專案  
2 ng new 專案名稱
```

💡 **Windows 快捷：** 在資料夾路徑列直接輸入 `cmd`，可直接在該目錄開啟命令提示字元。

建立專案 – 安裝選項

執行 `ng new` 後會出現兩個問題：

問題	選擇	說明
第一個選項 (樣式格式)	Sass (SCSS)	選擇 SCSS 作為樣式格式
第二個選項 (Server-Side Rendering)	N	輸入 N，不啟用 SSR

選完後等待安裝完成，看到成功畫面即代表專案建立完畢。

執行 Angular 專案

進入剛建立的專案目錄，執行 `ng serve` 啟動開發伺服器：

```
1 # 進入專案目錄
2 cd 專案名稱
3
4 # 啟動專案
5 ng serve
6
7 # 啟動並自動開啟瀏覽器（簡寫）
8 ng s --open
```

💡 預設網址：開啟瀏覽器後前往 `http://localhost:4200`，看到 Angular 預設頁面即代表成功。

3. 認識專案結構

Project Structure

專案根目錄 – 重要檔案

用 VS Code 開啟專案資料夾後，初期只需了解以下三個：

檔案 / 資料夾	說明
<code>node_modules/</code>	存放所有套件（底層與新增套件都在這）
<code>package.json</code>	記錄專案使用的套件與版本，可在此指定版本
<code>src/</code>	存放網站內容（HTML、SCSS、TS 等原始碼）

src 資料夾 – 核心檔案

檔案	說明
<code>styles.scss</code>	全站 Global CSS 設定
<code>main.ts</code>	整個網站的啟動入口點
<code>app.component.html</code>	使用者看到的畫面模板 (HTML)
<code>app.component.scss</code>	對應 HTML 的樣式 (SCSS)
<code>app.component.spec.ts</code>	測試案例撰寫檔案
<code>app.component.ts</code>	元件的邏輯程式碼 (Class)
<code>app.config.ts</code>	應用程式配置文件
<code>app.routes.ts</code>	路由配置文件

Angular 元件組成

一個 Angular 元件 (Component) 最少包含以下三個檔案，缺一不可：

檔案	職責
<code>.html</code>	畫面 UI 模板 (使用者看到的畫面)
<code>.scss</code>	UI 的樣式 (CSS)
<code>.ts</code>	程式碼邏輯 (Class)

💡 重點：三個檔案相輔相成，HTML 負責結構、SCSS 負責外觀、TS 負責邏輯，共同組成一個完整元件。

課程結束

環境建置完成，準備開始開發你的 **Angular** 應用程式！

ANGULAR ESSENTIALS

Angular 降版

「切換到 Angular 19，讓專案與套件版本相互對應」

[← 返回目錄](#)

Outline

- 1. 為什麼需要降版 — 常見情境與需求
- 2. 版本管理概念 — CLI、Core、RxJS、TypeScript 的對應關係
- 3. 降版流程 — 三步驟：確認版本、移除 CLI、重新安裝
- 4. 降版注意事項 — 套件相容性問題與處理方式

1. 為什麼需要降版？

Why Downgrade?

為什麼需要降版？

情境	說明
套件不相容	專案使用了與新版 Angular 不相容的套件
團隊架構考量	既有架構與 Angular 19 相容性較高
CI/CD 環境限制	建置環境尚未支援新版 Angular
避免升級成本	想避免不必要的重大版本升級風險
新版功能變更	特定功能在新版遭移除/變更，希望保留 Angular 19 行為

2. Angular 版本管理概念

Version Management

Angular 版本管理概念

概念	說明
套件相互對應	Angular CLI、Angular Core、RxJS、TypeScript 版本需彼此對應
全域 CLI 影響指令	全域安裝的 <code>@angular/cli</code> 決定 <code>ng</code> 指令的行為
專案版本以 <code>package.json</code> 為準	實際執行版本由專案的 <code>package.json</code> 決定
降版 = 三步調整	調整 CLI + 調整專案套件版本 + 重新安裝依賴

💡 關鍵：全域 CLI 與專案版本是獨立的，兩者都需要調整才能正確降版。

3. 降版流程

Downgrade Steps

步驟 1：確認目前版本

降版前先確認目前安裝的 Angular 版本：

```
1  ng version
```

💡 用途：確認全域 CLI 版本、Angular Core 版本、Node.js 版本等，作為降版前的基準記錄。

步驟 2：移除全域 Angular CLI

移除目前安裝的全域 Angular CLI：

```
1  npm uninstall -g @angular/cli
```

⚠ 注意：移除後 `ng` 指令暫時無法使用，需完成步驟 3 後才會恢復。

步驟 3：安裝 Angular 19 並建立專案

使用 `npx` 直接以指定版本建立新專案，無需重新全域安裝：

```
1 npx @angular/cli@19 new my-app-19
```

💡 說明：`npx` 會臨時下載並執行指定版本的 Angular CLI，專案建立後即可在 `package.json` 中確認版本是否正確。

package.json 版本確認 – dependencies

安裝完成後，確認 `package.json` 中的 `dependencies` 版本：

```
1 "dependencies": {  
2   "@angular/cdk": "^19.2.19",  
3   "@angular/common": "^19.2.0",  
4   "@angular/compiler": "^19.2.0",  
5   "@angular/core": "^19.2.0",  
6   "@angular/forms": "^19.2.0",  
7   "@angular/material": "^19.2.19",  
8   "@angular/platform-browser": "^19.2.0",  
9   "@angular/router": "^19.2.0"
```

package.json 版本確認 – dependencies (續) & devDependencies

```
1  "rxjs": "~7.8.0",  
2  "tslib": "^2.3.0",  
3  "zone.js": "~0.15.0"
```

```
1  "devDependencies": {  
2    "@angular-devkit/build-angular": "^19.2.19",  
3    "@angular/cli": "^19.2.19",  
4    "@angular/compiler-cli": "^19.2.0",  
5    "typescript": "~5.7.2"  
6  }
```

4. 降版注意事項

Known Issues

降版注意事項

問題類型	說明
套件相容性 (最常見)	第三方套件與降版後的 Angular 不相容
TypeScript / RxJS 版本	套件編譯時依賴特定 TS / RxJS 版本
Build pipeline 版本	<code>builder</code> / <code>angular.json</code> 設定與版本不符
Material / UI libs	部分 UI 套件不支援舊版 Angular
新功能反向不支援	使用到新版語法，降版後需調整 code

套件相容性問題

項目	說明
情境	降版後，第三方套件 (charts、auth、firebase、UI libs 等) 直接報錯或 build 失敗
原因	套件是為新版 Angular / TypeScript 編譯的，使用到 Angular 19 不支援的語法或 API

常見錯誤訊息：

```
1 This version of XXX requires Angular ≥20
2 NG0901: Unable to instantiate class...
```

套件相容性問題 – 處理方式

處理方式	說明
降版該套件	將第三方套件降版到與 Angular 19 相容的版本
查詢 release notes	找到官方說明的「Angular 19 支援區間」
替換 library	若無相容版本，需改用其他替代套件

💡 **建議流程：** 先查套件官方文件確認支援版本 → 嘗試降版 → 若無解，評估替換方案。

課程結束

掌握降版技巧，讓專案在正確的版本環境中穩定運行！

前端語言 HTML

「HTML & CSS — 樣板兄弟，前端之骨」

[← 返回目錄](#)

Outline

- HTML 介紹：什麼是 HTML？能做什麼？
- HTML 架構：文件結構拆解
- 標籤（元素）：行內 vs 區塊、屬性
- HTML 主要標籤總覽
- 常用標籤實作：H1-H6、列表、連結、圖片、表單
- 練習題 1 ~ 5

HTML 介紹

前端之骨

HTML 是什麼？

HTML (HyperText Markup Language · 超文本標記語言) 是一種用於建立網頁內容並呈現在瀏覽器上的標準標記語言。

- HTML 文件由一系列**標籤 (tags) **組成 (例如：按鈕、輸入框、文字、Table...等)
- 瀏覽器讀取 HTML 文件，並將其渲染成可視網頁
- 單純 HTML 不吸引人，需搭配 **CSS** 添加風格與布局，再加上 **JavaScript** 添加互動性

💡 記憶口訣：HTML 是骨架、CSS 是衣服、JavaScript 是動作

HTML 語法能做什麼

1. 結構化內容：HTML 定義網頁結構，將內容分為標題、段落、圖片等不同區塊
2. 支援多媒體：可嵌入圖片、音樂、影片等多媒體元素，提升網頁互動性
3. 超連結：透過 `<a>` 標籤在網頁間建立超連結，實現網頁跳轉
4. 表單提交：提供 `<form>` 元素讓使用者輸入資料並提交到伺服器
5. 表格展示：利用 `<table>` 標籤顯示資料表格
6. 支援樣式與設計：透過 CSS 定義網頁樣式和佈局
7. 與 JavaScript 互動：結合 JavaScript 實現動態功能和用戶互動
8. SEO 支援：`<title>` 和 `<meta>` 標籤有助搜尋引擎優化，提升網站可見度
9. 語義化標籤：HTML5 引入 `<header>`、`<article>` 等語義化標籤，使結構更清晰
10. 跨平台支援：網頁標準，支援各種設備和瀏覽器，保證內容一致呈現

HTML 架構

Document Structure

HTML 基本架構

- 普遍預設網頁入口都是檔名 `index` 為主
- 語法編輯概念和樂高相似，利用**標籤（元素）**來堆疊建構
- `<body>` 內層結構介紹

```
1  <!DOCTYPE html>
2  <html lang="en">      <!-- 網頁語系 -->
3    <head>
4      <meta charset="UTF-8">      <!-- 網頁編輯與拆解碼格式 -->
5      <meta name="viewport" content="width=device-width, initial-scale=1.
0">
6      <title>Document</title>    <!-- 網頁分頁名稱 -->
7    </head>
8    <body>
9      <!-- 網頁主體內容 -->
1
0    </body>
1
1  </html>
```

標籤（元素）

Tags & Elements

標籤（元素）概念

- `<` 稱標記； `<h1>` 稱標籤； `</h1>` 稱結尾標籤
- 標籤中可以涵蓋其他標籤（巢狀結構）
- 標籤有行內與區塊兩種元素型態：
 - 行內：不自動換行、左右空間不填滿
 - 區塊：自動換行且左右空間會主動填滿
- 能被分班級（ `class` ）、擁有識別證（ `id` ）、不同造型（ `style` ）以及擁有名字（ `name` ）

行內 vs 區塊 – 程式碼對照

```
1 <main>
2   <p class="block">Block 1</p>
3   <p class="block">Block 2</p>
4   <span name="Inline">Inline 1</span>
5   <span style="color: brown;">Inline 2</span>
6   <small id="small">call me small</small>
7 </main>
```

Block 1

Block 2

Inline 1 **Inline 2** call me small

💡 重點：Block 各占一行；Inline 元素排在同一行不換行

HTML 主要標籤

Tags Reference

主要標籤（一）－ 文件結構

標籤	說明
<code><!DOCTYPE html></code>	宣告文件為 HTML5，告訴瀏覽器用 HTML5 標準解析
<code><html></code>	HTML 頁面的根元素，所有標籤都放在這裡面
<code><head></code>	包含關於網頁的資訊， 不會在網頁上顯示
<code><title></code>	設定網頁標題，顯示在瀏覽器分頁上
<code><meta></code>	設定元數據，例如編碼、描述、關鍵字等
<code><link></code>	連結外部資源，例如 CSS 檔案
<code><body></code>	顯示網頁主要內容，使用者在瀏覽器中看到的部分
<code><h1></code> ~ <code><h6></code>	標題標籤，從 h1 (最大) 到 h6 (最小)

主要標籤（二）－文字、連結與列表

標籤	說明
<code><p></code>	段落標籤，用於顯示段落文字
<code><a></code>	連結標籤，用來建立超連結
<code></code>	圖像標籤，用來插入圖片
<code></code> & <code></code>	無序列表， <code></code> 是容器， <code></code> 是列表項目
<code></code> & <code></code>	有序列表， <code></code> 是容器， <code></code> 自動加上編號
<code><div></code>	區塊元素，用於分隔頁面區域，常用於布局
<code></code>	行內元素，對文字或行內元素進行分組
<code><table></code>	表格標籤，用於建立表格

主要標籤（三）－ 表單與語義化

標籤	說明
<code><form></code>	表單標籤，用來收集使用者輸入資料
<code><input></code>	輸入欄位，定義文字輸入、按鈕、選單等
<code><button></code>	按鈕標籤，定義可點擊的按鈕
<code><select></code> & <code><option></code>	下拉選單標籤
<code><iframe></code>	內嵌框架，在頁面中嵌入另一個 HTML 網頁
<code><nav></code>	導航標籤，定義頁面中的導航區塊
<code><header></code>	頁眉標籤，通常包含網站標題、導航
<code><footer></code>	頁腳標籤，定義頁面的尾部內容

常用標籤實作

Hands-on

編寫基本 HTML (練習)

開始編寫 HTML 只需一個文字編輯器 (Notepad 、 Sublime Text 、 VS Code 等) 。 也可用線上即時 HTML 編輯器：

<https://html.cafe/>

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>我的第一頁網頁</title>
5    </head>
6    <body>
7      <h1>歡迎來到我的網頁！</h1>
8      <p>這是一段簡單的文字。</p>
9    </body>
10  </html>
```

HTML 常用標籤 – H1~H6 & ul/ol

```
1 <h1>h1</h1> <h2>h2</h2> <h3>h3</h3>
2 <h4>h4</h4> <h5>h5</h5> <h6>h6</h6>
3
4 <ul>
5   <li>one</li> <li>two</li> <li>three</li>
6 </ul>
7
8 <ol>
9   <li>one</li> <li>two</li> <li>three</li>
10 </ol>
```

h1

h2

h3

h4 / h5 / h6

- one
 - two
 - three
1. one
 2. two
 3. three

HTML 常用標籤 – a、img

```
1 <main>
2   <a href="https://www.dcard.tw/f/whysoserious/p/235767148">前往酷酷的地方
</a>
3   
4 </main>
```

[前往酷酷的地方](https://www.dcard.tw/f/whysoserious/p/235767148)



💡 **img alt** : 圖片無法載入時顯示的替代文字，也有助於無障礙與 SEO

HTML 常用標籤 – textarea、div

```
1 <main>
2   <textarea name="" id="" cols="30" rows="10">Cool Der</textarea>
3   <div>
4     Hot Div
5   </div>
6 </main>
```

Cool Der

Hot Div

//

💡 **textarea** 是多行文字輸入框；**div** 是最常用的區塊容器，預設不帶樣式

輸入 (input) 標籤

- 輸入標籤經常被使用到，因為能輸入的型態 (type) 相當多元
- Type 有：**text** (文字)、**number** (數字)、**radio** (單選方塊)、**checkbox** (複選方塊)、**file** (選取檔案)、**password** (密碼) 等等

type	說明	顯示效果
text	一般文字輸入	文字框
number	數字輸入	數字框 (可按箭頭調整)
radio	單選方塊	圓形選項鈕 ●
checkbox	複選方塊	方形勾選框 □
file	選取檔案	選擇檔案按鈕
password	密碼輸入	隱藏字元顯示 ●●●●

輸入 (input) 標籤 – 範例

```
1 <label for="">Text</label> <input type="text" />
2 <label for="">Number</label> <input type="number" />
3 <label for="">Radio</label> <input type="radio" />
4 <label for="">Checkbox</label> <input type="checkbox" />
5 <label for="">file</label> <input type="file" />
6 <label for="">password</label> <input type="password" />
```

Text

Number

Radio



Checkbox



file

Choose File

No file chosen

password

Button & label 標籤

```
1 <label for="">列表名稱</label>
2 <button type="button">這是Button</button>
3 <button type="submit">這是submit</button>
```

列表名稱

這是Button

這是submit

💡 **type="button"**：一般按鈕，點擊不會提交表單；**type="submit"**：會觸發 **<form>** 的送出動作

練習題

HTML 練習

練習 1

請開啟專案並且試著將下方圖片的畫面做出來 (單純 HTML 無 CSS) 。

目標畫面：

HTML練習

 提示：只需要一個 `<h1>` 標籤，內容為「HTML練習」

練習 2

請開啟專案並且試著將下方圖片的畫面做出來 (單純 HTML 無 CSS) 。

目標畫面：

我是文字輸入框(可拉大)

-
- ☐
- 1. ☐ 選擇檔案 未選擇任何檔案
- 2. ☐

我是按鈕Button

練習 3

請開啟專案並且試著將下方圖片的畫面做出來 (單純 HTML 無 CSS) 。

目標畫面：

HTML練習

可愛的圖片



 提示：結構為 `<h1>` + `<p>` + `<img>` · 將圖片放入 `src` 屬性即可

練習 4

請開啟專案並且試著將下方圖片的畫面做出來 (單純 HTML 無 CSS) , 可參考
https://www.w3schools.com/html/html_tables.asp 。

目標畫面：

HTML練習

姓名	數學	英文
小明	90	85
小華	70	95

練習 5（上）－ 個人名片頁面

請開啟專案並且試著將下方圖片的畫面做出來（單純 HTML 無 CSS）。

王小明

嗨，我是一個正在學習 HTML 的新手！



關於我

我喜歡寫程式、看漫畫，還有玩電腦遊戲。

我的興趣

- 學習網頁設計
- 打籃球
- 旅行

聯絡我

Email：test@example.com

喜歡的網站：[Google](https://www.google.com)

我的作品

專案名稱	簡介
個人名片網頁	用純 HTML 做的第一個作品！
待完成的專案	持續增加中...

留言給我

姓名：

留言：

練習 5（下） – 完整頁面所需標籤

區塊

使用標籤

姓名 + 自我介紹

`<h2>`

`<p>`

`<img>`

關於我、我的興趣

`<h3>`

`<p>`

`<ul>`

`<li>`

聯絡我 + 連結

`<h3>`

`<p>`

`<a>`

我的作品

`<table>`

`<tr>`

`<th>`

`<td>`

留言給我

`<form>`

`<input>`

`<textarea>`

`<button>`

HTML 學習完成

掌握骨架，前端世界從這裡開始！

安裝 VS Code

「開發工具就位，前端開發正式啟動」

[← 返回目錄](#)

Outline

- 什麼是 VS Code ?
- 下載與安裝 VS Code
- 安裝 Angular Extension Pack 套件

什麼是 VS Code ?

Visual Studio Code

什麼是 VS Code ?

VS Code 全名 **Visual Studio Code**，是微軟開發的免費、跨平台程式碼編輯器。



特性	說明
免費 & 開源	完全免費，原始碼公開於 GitHub
跨平台	支援 Windows、macOS、Linux
豐富套件	擁有大量擴充套件可安裝
內建工具	Debug 工具、Git 整合、自動排版、IntelliSense

💡 為什麼選 **VS Code** ? 輕量、啟動快、套件生態豐富，是目前前端開發的主流編輯器

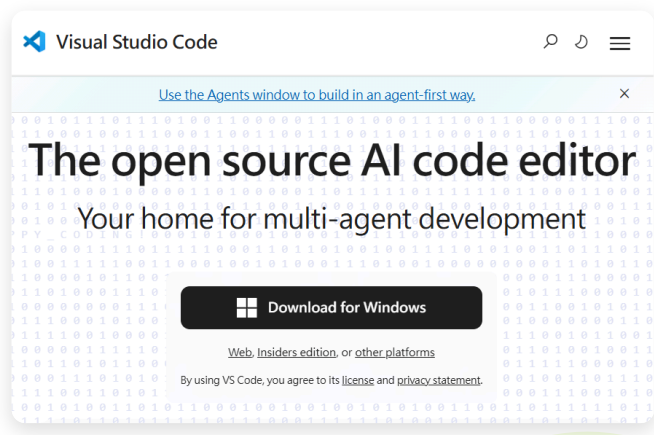
下載與安裝

Download & Install

Step 1 – 前往官網下載

開啟瀏覽器，前往 VS Code 官方網站：

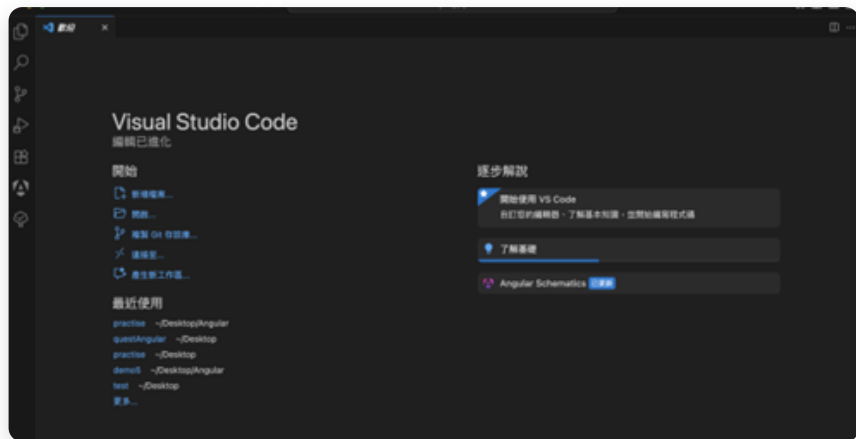
<https://code.visualstudio.com/>



💡 網頁會自動偵測你的作業系統並顯示對應版本；若未自動偵測，請手動選擇 Windows / macOS / Linux

Step 2 – 安裝完成後開啟

下載安裝檔後依指示完成安裝，開啟 VS Code 會看到歡迎頁面。



💡 第一次開啟會看到歡迎頁面，左側 Activity Bar 有檔案、搜尋、Git、Debug、套件等五大功能區

安裝 Angular 套件

Install Angular Extension Pack

Step 3 – 開啟延伸模組面板

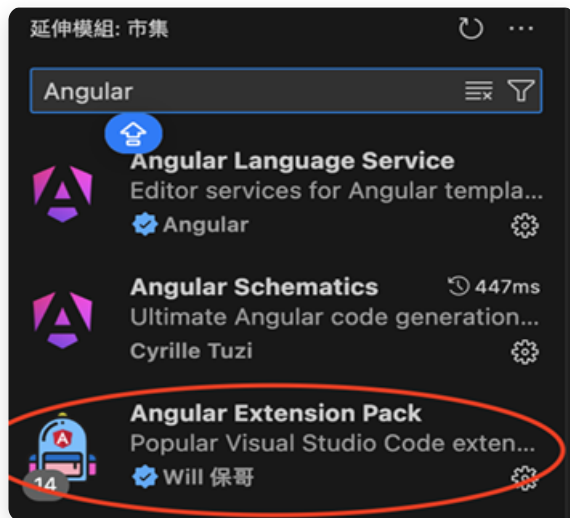
要安裝套件，請點擊左側 Activity Bar 中長得像方塊的圖示（延伸模組）。



💡 快速鍵：Windows **Ctrl + Shift + X**、macOS **⇧⌘X** 可直接開啟延伸模組面板

Step 4 – 搜尋 Angular 套件

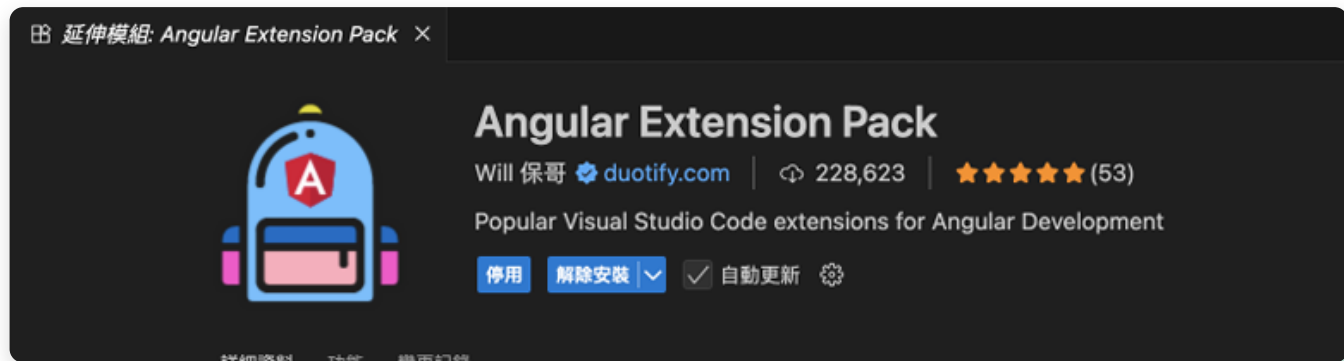
在搜尋欄輸入 **Angular**，在結果清單中找到製作者為 **Will 保哥** 的套件。



⚠ 注意：請選擇「**Angular Extension Pack**」，製作者為 **Will 保哥**，不要裝錯成其他同名套件

Step 5 – 點擊安裝

點擊 **Angular Extension Pack** 後，右側會出現套件詳細頁面，點擊「安裝」按鈕。



💡 安裝完成後按鈕會變成「停用 / 解除安裝」，代表已成功安裝

安裝完成！

完成以下兩個步驟後，VS Code 就設定好了：

步驟	操作	狀態確認
① 安裝 VS Code	從官網下載並安裝	可正常開啟 VS Code
② 安裝套件	搜尋並安裝 Angular Extension Pack	套件按鈕顯示「停用/解除安裝」

✅ 恭喜！開發環境已就緒，接下來可以開始撰寫 Angular 專案了！

安裝完成

工具就位，開始你的 **Angular** 開發之旅！

ANGULAR FULL-STACK MASTERCLASS

CSS 基礎語法

「前端之肉 – 讓網頁穿上衣服」

[← 返回目錄](#)

Outline

- CSS 常用屬性
- **Inline Style** — 在標籤上直接寫 CSS
- **CSS 選擇器** — 元素選擇器與 **Class** 選擇器
- **Class** 命名慣例
- **CSS 單位與 Box Model**
- 練習

CSS

前端之肉

CSS 常用屬性 (一)

屬性	說明
<code>color</code>	字體顏色
<code>font-size</code>	字體大小
<code>text-align</code>	文字對齊 (<code>left</code> 、 <code>right</code> 、 <code>center</code> 、 <code>end</code>)
<code>text-decoration</code>	文字裝飾 (<code>underline</code> 底線 、 <code>line-through</code> 刪除線)
<code>letter-spacing</code>	文字間距

CSS 常用屬性 (二)

屬性

說明

width

寬度

height

高度

border-radius

外框圓弧效果

box-shadow

陰影效果

Inline Style

在標籤上直接寫 CSS

建立 CSS 練習檔

先建立一個 HTML 檔案，在 `<body>` 中加入一個 `<p>` 標籤：

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>我的第三個網頁</title>
5  </head>
6  <body>
7    <p>CSS練習</p>
8  </body>
9  </html>
```

💡 開啟瀏覽器後，畫面上會出現 **CSS練習** 文字，目前尚未套用任何樣式

加入 Inline Style – 單個屬性

在 `<p>` 標籤上加入 `style` 屬性，直接寫入 CSS：

```
1 <p style='color:red;'>CSS練習</p>
```

💡 語法格式：`style='屬性名稱：值；'`，每個屬性結尾加 `；`

加入 Inline Style – 多個屬性

多個屬性寫在同一個 `style` 中，屬性之間用 `;` 分隔：

屬性	值	效果
<code>color</code>	<code>red</code>	文字變紅色
<code>font-size</code>	<code>30px</code>	文字放大至 30px

```
1 <p style='color:red; font-size: 30px; '>CSS練習</p>
```


CSS 選擇器

Selector

元素選擇器 (Element Selector)

若相同樣式要套用到所有相同標籤，使用 `<style>` 標籤宣告元素選擇器：

```
1 <style>
2   p { color: red; }
3 </style>
4 <body>
5   <p>CSS練習一</p>
6   <p>CSS練習二</p>
7   <p>CSS練習三</p>
8 </body>
```

⚠ 注意：元素選擇器 `p { }` 會修改所有 `<p>` 標籤的樣式！

Class 選擇器 — 概念

只修改部分標籤時，使用 Class 選擇器：

步驟	操作	說明
①	在 HTML 標籤加上 <code>class="pCss"</code>	<code>pCss</code> 為自訂選擇器名稱
②	在 <code><style></code> 中寫 <code>.pCss { }</code>	Class 選擇器前面必須加 <code>.</code> (點)

💡 要修改全部標籤用元素選擇器 (無 `.`) ; 只修改部分標籤用 Class 選擇器 (加 `.`)

Class 選擇器 – 範例

```
1 <style>
2   .pCss { color: red; }
3 </style>
4 <body>
5   <p class="pCss">CSS練習一</p>
6   <p>CSS練習二</p>
7   <p class="pCss">CSS練習三</p>
8 </body>
```

💡 練習一和練習三套用 **.pCss**（紅色）；練習二沒有 class，不受影響

多個 Class

一個標籤可以套用多個 **class**，中間用空格隔開：

```
1 <style>
2   .pCss { color: red; }
3   .pCss2 { font-size: 50px; }
4 </style>
5 <body>
6   <p class="pCss pCss2">CSS練習一</p>
7   <p class="pCss">CSS練習三</p>
8 </body>
```

💡 練習一同時套用 **.pCss**（紅色）與 **.pCss2**（50px 字體大小）

CSS 層疊規則 – 後蓋前

同一標籤套用多個 class，若修改相同屬性，後面的 class 會覆蓋前面的：

```
1 <style>
2   .pCss { color: red; }
3   .pCss2 { color: yellow; }
4 </style>
5 <body>
6   <p class="pCss pCss2">CSS練習一</p>
7 </body>
```

⚠ 結果：文字顏色為黃色 (.pCss2 寫在後面，程式由上往下執行，後蓋前)

Class 命名慣例

為讓程式易讀易維護，class 名稱應具有意義並採用固定格式：

命名方式	格式	好的範例	不好的範例
小駝峰 (camelCase)	第二字起大寫	<code>topText</code> 、 <code>loginBtn</code>	<code>area1text</code>
Kebab-case	用 <code>-</code> 連接	<code>top-text</code> 、 <code>login-btn</code>	<code>area1</code>

```
1 <div class="top"><div class="topText"></div></div>
```

單位與 Box Model

CSS Units & Box Model

CSS 單位

單位

說明

px

像素，網頁的基本固定單位

rem

相對單位， $1\text{rem} = 16\text{px}$ (根元素字體大小)

vw

視窗寬度的 1% · **100vw** = 全寬

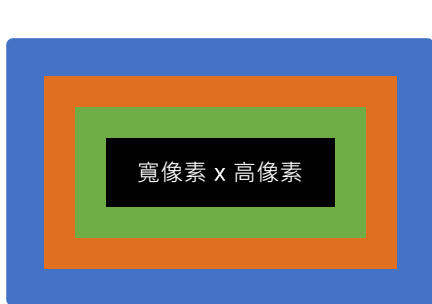
vh

視窗高度的 1% · **100vh** = 全高

```
1 width: 10vw;  
2 height: 10vh;  
3 margin: 1rem;
```

CSS Box Model – 四個元素

CSS Box Model 定義了每個 HTML 元素所佔的空間，由內而外共四層：



Content : CSS 設定的寬高



Padding : 元素與元素內容的距離，內留白



Border : 元素邊界



Margin : 元素與元素間的距離，外留白

CSS Box Model – Content（內容）

Content 為區塊的**主要內容**，像是文字、圖片、影片等，位於 Box Model 的**最內層**。大小由文字內容的多寡或圖片的長寬決定。

HTML練習

💡 藍色區域即為 Content；設定 `width` 和 `height`，就是在控制此區域的大小

CSS Box Model – Padding（內距）

Padding 是元素內部的空間距離，位於 Content 和 Border 之間。若有設定背景顏色，背景色會隨著 **Padding** 延伸。



HTML練習

💡 綠色外框 = Padding 區域（元素背景色延伸）；白色內框 = Content；可分別設定上下左右四個方向

CSS Box Model – Border（邊框）

Border 是元素的邊框，位於 Padding 外側。通常 1-5px 細線條，用來畫出元素邊界。可設定樣式（**solid** 實心 / **dashed** 虛線）、顏色與粗細。

HTML練習

```
1 border: 2px solid black;
```

CSS Box Model – Margin（外距）

Margin 是元素**外部**的空間距離，位於 Box Model 的**最外層**，用來控制元素與元素之間的距離。

HTML練習一

HTML練習二

⚠️ 兩個元素之間的空白 = Margin；背景色**不會**延伸至 Margin（與 Padding 的最大差異）

CSS Box Model — 範例

```
1  .boxModel {  
2    width: 10vw;  
3    height: 10vh;  
4    margin: 1rem;  
5    padding: 100px;  
6    border: 1px solid black;  
7    background-color: rgb(248, 203, 203);  
8  }
```

💡 border 語法： **border: 粗細 樣式 顏色** · 例如 **1px solid black** (1像素實心黑線)

Padding 與 Margin 設置

Padding 和 Margin 支援多種縮寫語法 (兩者寫法完全相同) :

語法	說明
<code>padding: 2px;</code>	四邊都為 2px
<code>padding: 2px 3px;</code>	上下 2px、左右 3px
<code>padding: 2px 3px 4px;</code>	上 2px、右 3px、下 4px (左同右)
<code>padding: 2px 3px 4px 5px;</code>	上 右 下 左 (順時針)
<code>padding-top / right / bottom / left</code>	單獨設定各方向

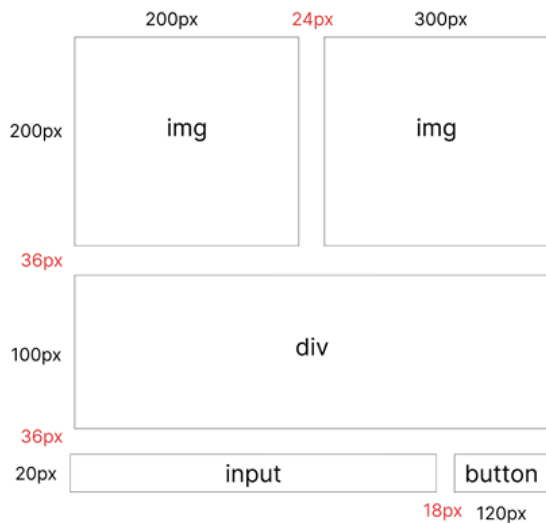
練習

CSS Practice

練習 1 – Box Model 尺寸設計

任務說明

根據線框圖建立 HTML 結構，並套用 CSS 設定各元素的寬高與間距：



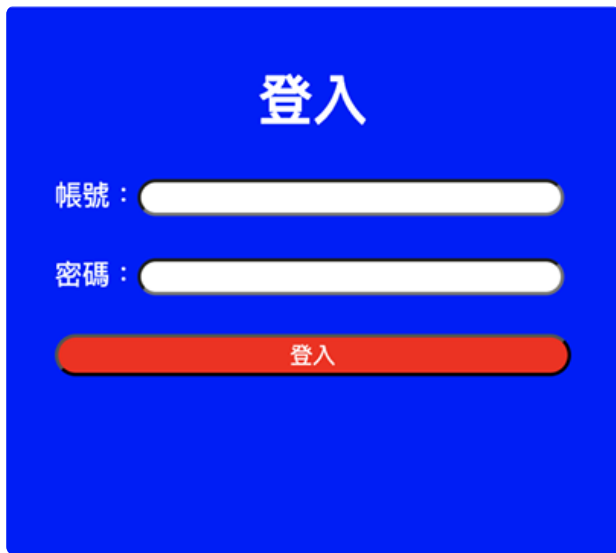
練習 1 – 解題提示

元素	屬性	值
img (左)	width / height / margin-right	200px / 200px / 24px
img (右)	width / height	300px / 200px
div	height / margin	100px / 36px 0
input	height	20px
button	width / height / margin-left	120px / 20px / 18px

練習 2 – 登入頁面

任務說明

使用 HTML + CSS 製作下方登入頁面：



登入

帳號：

密碼：

登入

練習 2 – 解題提示

區塊	使用標籤	CSS 重點
外框	<code>div</code>	<code>background-color: blue</code>
標題	<code>h1</code> "登入"	<code>color: white; text-align: center</code>
帳號 / 密碼標籤	<code>p</code>	<code>color: white</code>
帳號 / 密碼輸入框	<code>input</code>	<code>border-radius: 999px</code>
登入按鈕	<code>button</code>	<code>background-color: red; color: white; border-radius: 999px</code>

CSS 基礎完成

讓你的網頁穿上第一件衣服！

ANGULAR FULL-STACK MASTERCLASS

CSS 樣式編輯

「互動、排版、套件整合」

[← 返回目錄](#)

Outline

- CSS 互動 — :hover 與 :active
- display: flex — 主軸與次軸
- 對齊 — justify-content 與 align-items
- 安裝 Bootstrap
- 練習

CSS 互動

:hover 與 :active

CSS 互動－概念

前端標籤樣式可以設定成**互動式樣式**，根據使用者操作產生不同的視覺回饋：

偽類	觸發條件	說明
<code>.class:hover</code>	滑鼠移至標籤	滑鼠懸停時的樣式回饋
<code>.class:active</code>	滑鼠點擊時	按下滑鼠時的樣式回饋

💡 SCSS 寫法：`.class { &:hover { } } }`，注意 `&:hover` 在 SCSS 中算是子層

💡 搭配 `transition` 屬性調整互動時間，能大幅提升使用質感

:hover — 滑鼠移至標籤

```
1  .topText {  
2    width: 100px;  
3    height: 100px;  
4    transition: 0.8s;  
5    background-color: aqua;  
6    &:hover {  
7      background-color: #888;  
8    }  
9  }
```

原本的顏色



滑鼠移至目標標籤時



:active — 滑鼠點擊標籤

```
1  .topText {  
2    width: 100px;  
3    height: 100px;  
4    transition: 0.8s;  
5    background-color: aqua;  
6    &.active {  
7      background-color: #333;  
8    }  
9  }
```

原本的顏色



滑鼠點擊目標標籤時



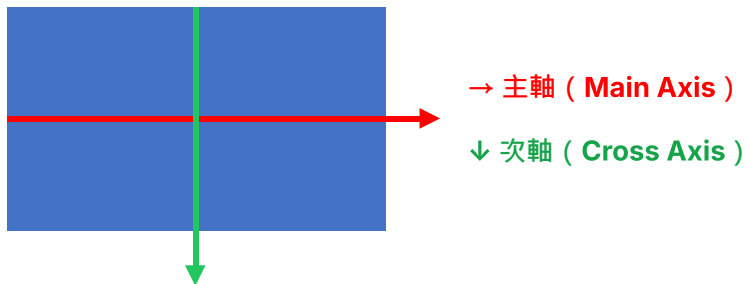
display: flex

主軸與次軸

display: flex – 介紹

display: flex 讓容器擁有主軸與次軸，並改變子元素的排列方式：

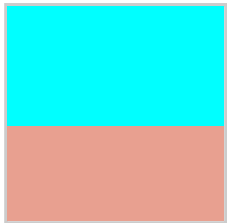
- 標籤元素都變成區塊元素 (Block)
- 主軸 (Main Axis)：預設由左往右 →
- 次軸 (Cross Axis)：與主軸垂直，由上往下 ↓



display: flex – 主次軸比較

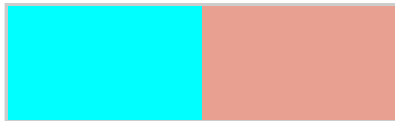
無 display: flex

主軸 ↓ 由上往下 (無次軸)



有 display: flex

主軸 → 由左往右 | 次軸 ↓



💡 參考文件 : bootstrap5.hexschool.com/docs/5.1/utilities/flex/

對齊

justify-content & align-items

對齊－概念

display: flex 啟用後，可以搭配以下兩個屬性控制對齊方式：

屬性	軸向	說明
justify-content	主軸	控制子元素在主軸方向上的對齊方式
align-items	次軸	控制子元素在次軸方向上的對齊方式

💡 **vertical-align**：只適用於行內元素，以基準線方式對齊；用於 **img** 標籤可取消預設底部留白

justify-content – 主軸對齊

無 justify-content



justify-content: center



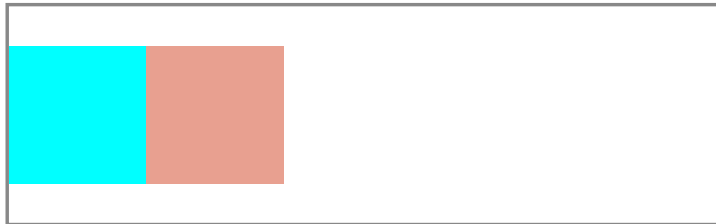
```
1 display: flex;  
2 justify-content: center;
```

align-items – 次軸對齊

無 align-items



align-items: center



```
1 display: flex;  
2 align-items: center;
```

安裝 Bootstrap

Bootstrap Integration

安裝 Bootstrap – 步驟

步驟

操作

- ① 在終端機 (專案根目錄) 執行 `npm i bootstrap`
- ② 安裝完成後開啟 `angular.json`
- ③ 找到 `"styles": [ ... ]` · 在第一個位置加入 Bootstrap CSS 路徑
- ④ 重啟專案

```
1 "styles": [  
2   "./node_modules/bootstrap/dist/css/bootstrap.min.css",  
3   "src/styles.scss"  
4 ],
```

安裝 Bootstrap — 原理

`npm i bootstrap` 的用途是將 Bootstrap 套件安裝進專案，就像 HTML 的 `<link>` import 一樣（差別在於 `npm i` 是安裝套件進專案中）。

在 `angular.json` 的 `styles` 加入套件路徑，等同於告訴專案「我要匯入這個 CSS，並告訴它 CSS 的位置」。

💡 安裝後的套件都存放在 `node_modules/` 資料夾，路徑以 `./node_modules/` 開頭

練習

CSS Practice

練習 – Flex 排版

任務說明

試著做出下圖的排版：

- 中間間隔為 **10px**
- Flex item 3 左右各間隔 **10px**
- Flex item 1 跟 5 高度為 **80px**



CSS 樣式編輯完成

互動、排版、套件，全部就位！

ANGULAR FULL-STACK MASTERCLASS

CSS 進階工具

「背景、定位、堆疊，掌控版面細節」

[← 返回目錄](#)

Outline

- 背景設定 — `background-image` / `repeat` / `position` / `size`
- `Position` — `fixed` / `relative` & `absolute`
- `z-index` — 堆疊順序
- 後蓋前觀念 — `CSS` 優先權
- 練習

背景設定

Background Properties

背景設定－屬性總覽

background-* 系列屬性可精細控制元素的背景圖片顯示方式：

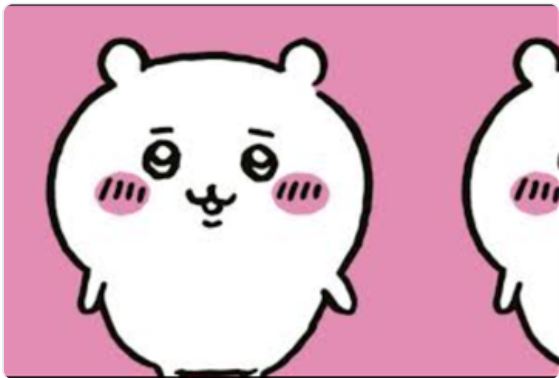
屬性	說明
background-image	設定背景圖片，使用 url(...) 指定路徑
background-repeat	控制背景是否重複平鋪，常用 no-repeat
background-position	設定背景位置，格式為 X Y (水平 垂直)
background-size	指定背景圖片大小 (px 、 % 、 cover 、 contain)

💡 **background-position** 前方數值代表水平位置 (X)，後方代表垂直位置 (Y)

background-image – 本地端圖片

`url()` 可以放入本地端 (電腦) 圖片的路徑：

```
1 background-image: url("../IMG_8536(1).jpg");
```

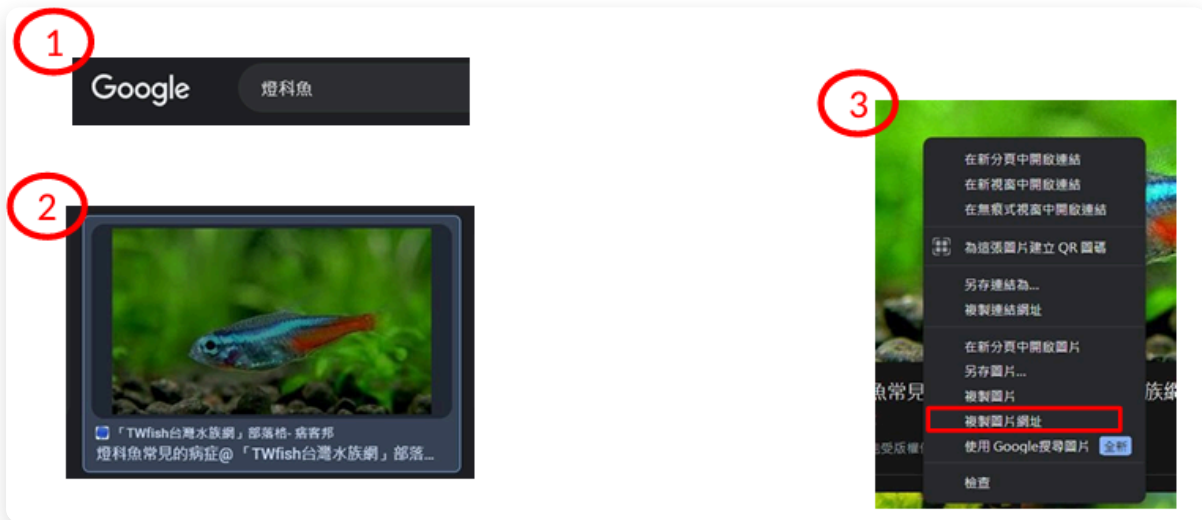


💡 路徑以 `./` 開頭代表相對路徑，與 CSS 檔案位於同一層目錄

⚠️ 圖片名稱含括號、空格等特殊字元時，記得確認路徑格式正確

background-image – 網路圖片：取得網址

`url()` 也可以放入網路圖片的網址，三步驟取得圖片網址：



⚠ 使用網路圖片前請注意版權，此處僅作練習用途

background-image – 網路圖片：範例

```
1 background-image: url("https://pic.pimg.tw/twfish0999/1361897489-2554509122_n.jpg");
```



💡 直接貼上圖片的完整 URL，效果與本地端相同

預設情況下背景圖片會重複平鋪填滿整個元素

background-repeat — 背景是否重複

預設背景圖片會重複平鋪，加上 `no-repeat` 可取消重複：

```
1 background-repeat: no-repeat;
```

預設 (重複平鋪)



`no-repeat` (不重複)



background-position – 設定背景位置

格式 **x y**：前方數值為水平位置，後方為垂直位置：

```
1 background-repeat: no-repeat;  
2 background-position: 50% 50%;
```



💡 X / Y 數值對照：

X： 0% = 靠左， 50% = 置中， 100% = 靠右

Y： 0% = 頂端， 50% = 置中， 100% = 底端

50% 50% 表示圖片顯示在正中央

background-size — 設定背景圖片大小

值

說明

px / %

直接指定圖片寬高

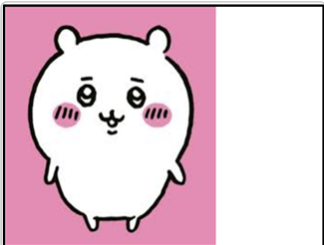
cover

等比放大至填滿容器，可能裁切；解析度低時易失真

contain

等比縮放至可完整放入容器，不裁切

```
1 background-repeat: no-repeat;  
2 background-size: contain;
```



Position (位置)

fixed / relative / absolute

position: fixed – 定錨

position: fixed 將標籤固定在視窗的絕對位置，不隨滾輪移動：

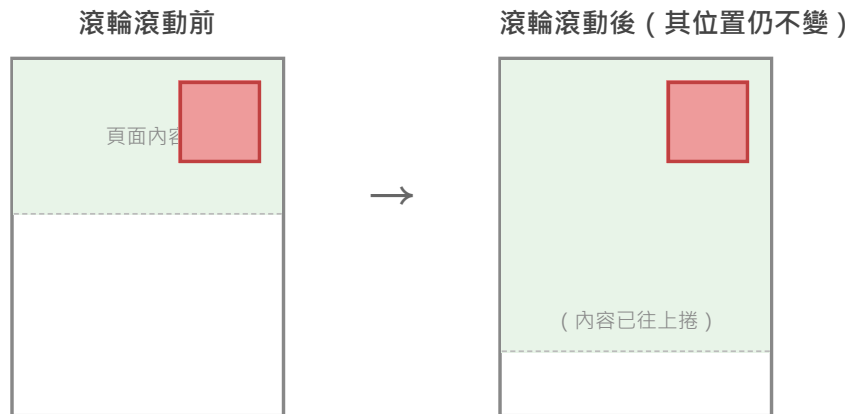
- 定位方式以視窗的上下左右為基準
- 不論如何縮放網頁或滾動滾輪，位置始終不變
- 搭配 **top**、**right**、**bottom**、**left** 指定距視窗邊緣的距離

```
1 .topContent {  
2   position: fixed;  
3   top: 20px;  
4   right: 50px;  
5 }
```

💡 常用於固定導覽列、懸浮按鈕、回頂端按鈕等不隨頁面捲動的元件

position: fixed – 示意圖

滾輪滾動後，**fixed** 元素位置相對視窗保持不變：



position: relative & absolute – 父子關係定錨

relative 與 **absolute** 搭配使用，讓子標籤在父標籤範圍內進行定位：

角色	CSS 設定	說明
父標籤	position: relative	建立定位基準，本身位置不移動
子標籤	position: absolute	在父標籤區域內進行定錨

```
1 <div class="top">
2   <div class="topContent"></div>
3 </div>
```

💡 標籤 **topContent** 被 **top** 包住，即形成父子關係：父 → **top**，子 → **topContent**

position: relative & absolute – CSS 範例

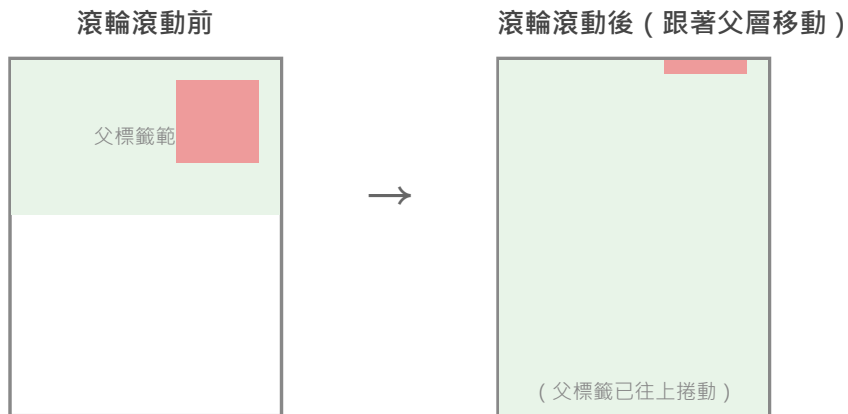
```
1 .top {  
2   width: 100vw;  
3   height: 110vh;  
4   position: relative;  
5 }
```

```
1 .topContent {  
2   width: 100px;  
3   height: 100px;  
4   background-color: #ee9b9b;  
5   position: absolute;  
6   top: 5%;  
7   right: 1%;  
8 }
```

💡 滾動頁面時，**absolute** 元素的位置會跟著父標籤移動（非固定在視窗）

position: relative & absolute – 示意圖

absolute 元素釘在父標籤的某一位置，滾動時跟著父標籤移動：



z-index

堆疊順序

z-index – 堆疊順序

z-index 設定元素的前後堆疊順序，數值越高越前面：

- 可設為負數（元素會疊在其他元素後方）
- 只對有設定 **position** 的元素有效
- 注意：**position: relative** 的父層本身無法移動



後蓋前觀念

CSS 優先權

後蓋前觀念

在 CSS 中，針對相同標籤的樣式設定，後方的規則會覆蓋前方的規則：

```
1 h1 { color: red; }  
2 h1 { color: blue; }
```

color: red → color: blue (後方覆蓋)

→ XXX

以這個 `h1` 標籤為例：

```
1 <h1 class="x1">XXX</h1>
```

當兩條規則都以**標籤名稱 (`h1`) **為選擇器時，後方規則生效。

後蓋前觀念 – class 選擇器

若指定 `.class` 名稱，不會受後方標籤設定的樣式影響：

```
1  .x1 { color: red; }  
2  h1  { color: blue; }
```

`.x1` → class 選擇器優先權高於標籤選擇器

→ **XXX**

💡 CSS 選擇器優先權 (Specificity) 由高至低：

`id (#id)` > `class (.class)` > `標籤 (h1, div)`

class 選擇器優先權高，不會被後方的標籤選擇器覆蓋

後蓋前小注意（一）

- ID

- 一般標籤除了設定 `class` 名稱，也會設定 `id` 名稱
- `id` 普遍用途在於方便 JavaScript 開發時取得指定值
- `id` 的特性在於不重複，偏向功能性開發用途，而非樣式設定

- 樣式鎖定（較不常用）

- 在樣式設定中加上 `!important`，即使後方有同標籤的規則，該樣式也不會被覆蓋
- 無必要情況時不添加

```
1 h1 { color: red !important; }  
2 h1 { color: blue; }
```

加了 `!important` 的 red 不會被 blue 覆蓋 → **XXX**

後蓋前小注意（二）

- 語法建議

- 接收他人編輯過的網頁時，建議以後蓋前的觀念建立新的 `.css` 檔案覆蓋舊檔
- 避免前人設計的樣式被更動後無法挽回

- 檔案管理

- CSS、TS 個別放，共用的放在一起

練習

CSS Practice

練習 – Position 定位排版

任務說明

使用 `position` 屬性，試著做出下方的三層色塊排版：

圖形規格：

- 深灰色 300px × 300px `#474a4d`
- 橘色 200px × 200px `#f7b977`
- 綠色 100px × 100px `#028760`

💡 更改顏色：`background-color: 色碼;`



練習 – Position 定位排版

解題提示

1. 深灰色為最外層父標籤，設定 `position: relative`
2. 橘色為深灰色的子標籤，設定 `position: absolute`，靠右上角 (`top: 0; right: 0`)
3. 綠色為橘色的子標籤，設定 `position: absolute`，靠左下角 (`bottom: 0; left: 0`)

```
1 .black { position: relative; width: 300px; height: 300px; }  
2 .orange { position: absolute; top: 0; right: 0; width: 200px; height: 200px; }  
3 .green { position: absolute; bottom: 0; left: 0; width: 100px; height: 100px; }
```

💡 記得在 HTML 中建立正確的巢狀結構： `black > orange > green`

CSS 進階工具完成

背景、定位、堆疊，全部就位！

ANGULAR FULL-STACK MASTERCLASS

JavaScript 介紹

「讓網頁動起來的語言」

[← 返回目錄](#)

Outline

- JavaScript 是什麼？
- TypeScript 是什麼？
- TypeScript VS JavaScript
- JavaScript 實例

JavaScript 是什麼？

What is JavaScript?

JavaScript 是什麼？

JavaScript 是一種腳本，也能稱它為**程式語言**，可以讓你在網頁中實現出複雜的功能。

當網頁不只呈現靜態的內容，另外提供了像是：

- 內容即時更新
- 地圖互動
- 繪製 2D / 3D 圖形
- 影片播放控制.....

你就可以大膽地認為 **JavaScript** 已經參與其中。

💡 網頁的三層架構：**HTML** (內容) → **CSS** (樣式) → **JavaScript** (互動)

JavaScript – 三層架構

HTML、CSS、JavaScript 分別負責網頁的不同層次：



JavaScript：負責互動與行為邏輯

CSS：負責視覺樣式與排版

HTML：負責頁面結構與內容

TypeScript 是什麼？

What is TypeScript?

TypeScript 是什麼？

TypeScript 是一種由 **Microsoft** 開發的開源程式語言，它是 JavaScript 的一種**超集 (Superset)**。

- 任何有效的 JavaScript 程式碼，也是有效的 TypeScript 程式碼
- TypeScript 為 JavaScript 新增了****類型系統 (型別) ****和其他功能：
 - **Interfaces**
 - **Generics**
 - **Enums**

這些功能使開發者能更有效地撰寫大型專案。

💡 **Angular** 就是使用 TypeScript 開發，所以要有一定的 JavaScript 基礎，才能更好地理解 TypeScript 的使用

TypeScript VS JavaScript

雖然 TypeScript 與 JavaScript 在許多方面都很相似，但它們之間仍存在一些重要的區別：

比較項目	JavaScript	TypeScript
類型系統	動態型別，變數類型可在執行期間改變	靜態型別，宣告時確定類型，有助於預先找出錯誤
編譯器	無需編譯，可在瀏覽器直接執行	需要編譯成 JavaScript 才能在瀏覽器執行
物件導向	有 <code>class</code> 與 <code>object</code> ，但無 <code>interface</code>	支援 <code>interface</code> 、 <code>class</code> 、 <code>object</code> ，物件導向更直觀

JavaScript 實例

JavaScript in Action

JavaScript 實例 – Step 1 : HTML

先在 HTML 中建立一個按鈕，加入 `class` 命名樣式名稱，並加入 `onclick` 方法（觸發動作的固定寫法）：

```
1  <!-- body 內是寫 HTML 內容 -->
2  <body>
3    <button class="bntCss" onclick="clickBnt()">我是按鈕!點我!</button>
4  </body>
```

畫面預覽：

我是按鈕!點我!



`onclick="clickBnt()"` 表示「點擊時，執行名為 `clickBnt` 的方法」，方法名稱可自訂

JavaScript 實例 – Step 2 : CSS

用剛才在 HTML 設定的 `class` 名稱，來編輯按鈕的樣式：

```
1  <!-- style 內是寫 CSS 內容 -->
2  <style>
3    .bntCss {
```